DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING • DEVOPS

Continuous Assessment II (CA-II): Case Study Evaluation Report

Architectural Fragility, Microservices Transition, and High-Velocity DevOps Engineering

GROUP	Group 33		
MEMBER 1	Kshitiz Shah (PRN: 23070122121)	MEMBER 2	Nitesh Ghimire (PRN: 23070122150)
MEMBER 3	Nigel Francy (PRN: 23070122148)	MEMBER 4	Omkar Kadam (PRN: 23070122158)

1. EXECUTIVE SUMMARY & OVERVIEW

This report presents a comprehensive academic evaluation of two seminal industry case studies in cloud computing, system architecture, and DevOps culture: **Netflix Inc.** and **Amazon.com Inc.** Both technology leaders faced catastrophic scalability ceilings, single points of failure, and deployment gridlock during their early growth phases while operating centralized monolithic architectures. Through a methodical deconstruction of the technical catalysts, architectural decisions (loosely coupled microservices and cloud infrastructure), and organizational restructuring (Two-Pizza teams and Chaos Engineering), this evaluation analyzes how these organizations transformed operational vulnerabilities into competitive, industry-defining engineering advantages.

2. CASE STUDY 1: NETFLIX INC. – HIGH-RESILIENCE CLOUD MIGRATION & CHAOS ENGINEERING

> **Case Study Scenario (Q1):** *In August 2008, a major relational database corruption halted Netflix's DVD shipments for three consecutive days, highlighting the systemic fragility of their monolithic architecture. Analyze the specific challenges faced by Netflix with its monolithic system, evaluate their transition to cloud-native microservices, and explain how the pioneering of Chaos Engineering transformed platform resilience and scalability.*

2.1 Specific Challenges Highlighted by the 2008 Database Outage

In 2008, Netflix operated on an on-premises monolithic system hosted in private datacenters, backed by a centralized relational database (Oracle RDBMS). The 72-hour operational paralysis caused by database table corruption revealed deep-seated structural vulnerabilities:

- **Catastrophic Single Point of Failure (SPOF):** The entire operational workflow—DVD shipping queue generation, inventory tracking, customer billing, and account management—shared a common database cluster. A single database corruption incident paralyzed all business operations simultaneously.
- **Tight Architectural Coupling and Unbounded Blast Radius:** In a monolithic codebase, disparate functional domains run within a shared execution process and share memory and datastore connections. A software fault or deadlocked query in one subsystem cascaded across the entire platform without fault containment boundaries.
- **Vertical Hardware Scalability Ceilings:** Datacenter monoliths could only scale vertically by purchasing progressively larger and more expensive enterprise servers. When Netflix began streaming video over the internet, extreme peak evening traffic surges regularly saturated physical datacenter bandwidth and compute capacity.
- **Deployment Risk & Release Coordination Overhead:** Code contributions from across the engineering team were merged into a single massive release artifact. Releases required prolonged manual regression testing, extensive coordination meetings, and high-risk maintenance downtime windows.

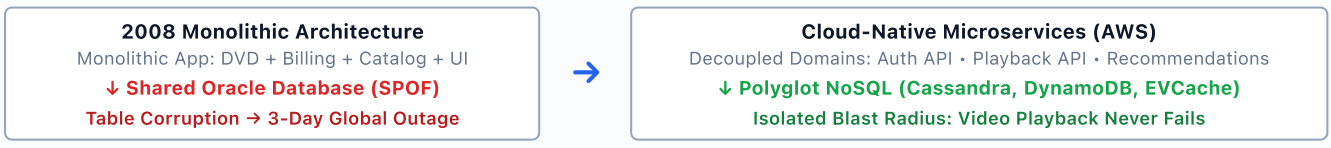


Figure 1: Architectural Transition from Monolithic Single Point of Failure to Cloud-Native Distributed Microservices at Netflix.

2.2 Architectural Transition: Microservices and AWS Cloud Adoption

Following the 2008 outage, Netflix initiated a multi-year strategic migration to Amazon Web Services (AWS), decomposing the monolith into a cloud-native microservices architecture:

- **Decoupled Microservice Boundaries:** The application was partitioned into hundreds of independently managed microservices (e.g., Discovery Service, Playback API, Recommendation Engine, Billing). Each service communicates strictly over lightweight RESTful HTTP/JSON interfaces.
- **Database-per-Service Paradigm:** Netflix decommissioned the centralized relational database, giving each microservice its own dedicated datastore. The architecture adopted distributed, horizontally scalable NoSQL systems (`Apache Cassandra`, `Amazon DynamoDB`, and `Amazon S3`) complemented by high-performance distributed caching (`EVCache`). A database issue in user reviews cannot disrupt subscriber playback authentication.

- **Immutable Infrastructure:** Virtual machine images (AMIs) are baked automatically through CI/CD pipelines using `Spinnaker` and `Aminator`. Infrastructure is treated as immutable—servers are never modified in-place; updates are deployed by provisioning fresh cloud instances and terminating old ones.
- **Automated Canary Deployments:** Using automated analysis tools (`Kayenta`), continuous delivery pipelines route 1–2% of production traffic to canary instances, algorithmically validating error rates and latency telemetry before completing fleet-wide rollout.

2.3 Pioneering Chaos Engineering: Proactive Anti-Fragility

In large-scale distributed cloud systems spanning thousands of virtual servers, localized component failures, disk corruptions, and network partitions are inevitable. Netflix concluded that attempting to prevent all failures was futile; instead, systems must be built to survive failure gracefully. This philosophy created the discipline of **Chaos Engineering**:

- **Guiding Philosophy:** *"The best way to avoid failure is to fail constantly."* Rather than waiting for outages to occur unexpectedly during peak hours, engineers deliberately inject controlled failures into production during standard business hours when engineering teams are actively monitoring systems.
- **The Simian Army Tool Suite:**
  - **Chaos Monkey:** Randomly terminates running Amazon EC2 instances in production to prove that application instances self-heal and load balancers redirect traffic without customer impact.
  - **Chaos Gorilla:** Simulates a catastrophic failure of an entire AWS Availability Zone (AZ), verifying automated multi-datacenter failover.
  - **Chaos Kong:** Simulates an entire AWS Region outage, validating global multi-region active-active failover via Route 53 and cross-region data replication.
  - **Latency Monkey / FIT (Failure Injection Testing):** Introduces artificial network latency and dropped packets between microservices to expose latent cascading timeout deadlocks.
- **Defensive Architectural Resilience Patterns:**
  - **Circuit Breakers (`Hystrix`):** When a downstream microservice exceeds pre-defined error or latency thresholds, circuit breakers trip open immediately, shielding thread pools from starvation.
  - **Graceful Degradation:** If personalized recommendation machine-learning algorithms fail or slow down, the presentation UI gracefully degrades to a cached list of top-ranked movies. The core business operation—allowing the customer to press "Play"—remains fully functional.

2.4 Measurable Resilience and Scalability Impact

By pairing distributed microservices with continuous chaos testing, Netflix eliminated centralized single points of failure. The architecture routinely delivers **99.999% availability ("five nines")** at planetary scale, supporting over 260 million global subscribers across 190+ countries while handling tens of millions of concurrent video streams with zero single-point dependency.

3. CASE STUDY 2: AMAZON INC. – DECENTRALIZED ARCHITECTURE, TWO-PIZZA TEAMS, AND CONTINUOUS DELIVERY

> **Case Study Scenario (Q2):** *In the early 2000s, Amazon's rapid growth was severely hampered by a monolithic architecture that led to frequent outages and slow feature releases, directly impacting customer satisfaction and revenue. Analyze how Amazon's adoption of 'Two-Pizza Teams' and a microservices architecture directly addressed these challenges, and explain how this cultural and architectural shift fostered the 'culture of continuous innovation' that allowed them to deploy code, on average, every 11.7 seconds.*

3.1 Monolithic Constraints: The "Obidos" Bottleneck

During its first decade of exponential commercial expansion, Amazon operated its retail website on a massive monolithic C/C++ application known internally as **"Obidos"** (named after a town on the Amazon River). Obidos coupled the user interface, shopping cart, product catalog, customer reviews, and order fulfillment workflows into a single executable binary.

- **Brooks' Law & Communication Drag:** As Amazon expanded its engineering headcount to meet commercial demand, team coordination overhead expanded quadratically ($N(N-1)/2$). Engineers spent the majority of their time in cross-functional synchronization meetings, dependency negotiations, and resolving git merge conflicts.
- **Deployment Queues ("Deployment Trains"):** Hundreds of engineers committed code into the monolithic Obidos repository. Releases ran on scheduled "trains." If one engineer committed code with a fatal regression or broken unit test, the entire deployment train was halted for all teams across the enterprise.
- **Unconstrained Blast Radius:** Because all application modules shared the same operating system process, a memory leak or null pointer exception in a non-essential feature (e.g., book review comments) could bring down the entire checkout engine, costing millions of dollars in lost transactional revenue.

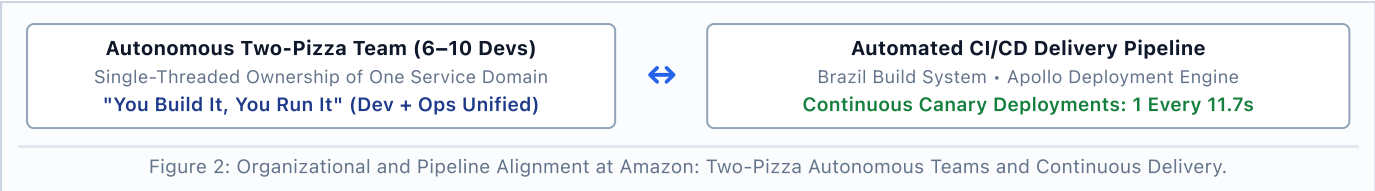


Figure 2: Organizational and Pipeline Alignment at Amazon: Two-Pizza Autonomous Teams and Continuous Delivery.

3.2 Architectural Decoupling: The 2002 Bezos "API Mandate"

Around 2002, CEO Jeff Bezos issued his famous architectural memorandum, known throughout the software engineering industry as the **API Mandate**. The memorandum established non-negotiable rules for all software development across Amazon:

1. All teams must expose their data and functionality exclusively through well-defined service interfaces (APIs).
2. Teams must communicate with each other solely through these interfaces.
3. No direct inter-process communication, shared memory, or direct database access across teams is permitted.
4. The interface technology must be completely platform-agnostic (HTTP, REST, XML/JSON).
5. Every interface must be architected, built, and tested so that it can be externalized to third-party developers without refactoring.

This mandate systematically decoupled Obidos into autonomous, service-oriented modules (Product Catalog, Shopping Cart, Pricing Engine, Payment Gateway). Furthermore, the discipline of building externalizable service APIs directly laid the architectural, technological, and infrastructural foundation for **Amazon Web Services (AWS)**.

3.3 Organizational Re-engineering: Conway's Law & "Two-Pizza Teams"

Amazon recognized that architectural transformation cannot succeed within a traditional, centralized organizational hierarchy. This realization is rooted in **Conway's Law**: *"Organizations which design systems are constrained to produce designs which are copies of the communication structures of these organizations."* To support decoupled microservices, Amazon executed a deliberate "Reverse Conway Maneuver":

- **Two-Pizza Team Topology:** Engineering teams were capped at a size that could be adequately fed by two pizzas—typically between 6 and 10 engineers. Small team sizes eliminate communication drag, eliminate coordination meetings, and allow rapid consensus.
- **Single-Threaded Ownership:** Each two-pizza team has end-to-end, single-threaded ownership over one discrete service or customer-facing capability (e.g., one team owns the "1-Click Ordering" service, another owns "Customer Review Moderation").
- **"You Build It, You Run It" (Werner Vogels Philosophy):** Operational silos between developers and system administrators were dissolved. Developers who write the service code are responsible for testing, automated deployment, monitoring, performance tuning, and carrying the 24/7 on-call operational pager. This direct operational accountability creates powerful incentives to write maintainable, resilient, and observable code.
- **Decentralized Autonomy:** Teams operate as autonomous internal business units. They choose the programming languages, database engines, and frameworks best suited for their specific domain, releasing features directly based on customer telemetry without centralized approvals.

3.4 Continuous Innovation and the 11.7-Second Deployment Rate

To enable thousands of autonomous two-pizza teams to deploy code independently without breaking the retail website, Amazon developed sophisticated internal DevOps automation:

- **Automated CI/CD Tooling (`Brazil` & `Apollo`):** Amazon built the `Brazil` build system and the `Apollo` deployment engine. Whenever an engineer commits code, the pipeline automatically compiles the artifact, executes comprehensive unit and integration test suites, performs automated security and compliance scans, and triggers progressive rollouts to isolated fleet cells.
- **Elimination of "Big-Bang" Risk:** Decomposing monolithic releases into continuous micro-deployments drastically lowered deployment risk. Because each deployment contains only a minimal code diff, the blast radius of any failure is tiny. If an automated metric alarm detects elevated error rates or latency degradation, Apollo automatically halts the canary rollout and reverts to the previous stable state within seconds.
- **High-Frequency Cadence (11.7 Seconds):** By reaching an average of **one production deployment every 11.7 seconds** across its microservices fleet, deployment was transformed from a stressful quarterly event into a routine, continuous background activity. This velocity empowers Amazon teams to conduct rapid A/B testing, gather real-time customer data, and iterate constantly without operational risk.

4. COMPARATIVE SYNTHESIS MATRIX

DIMENSION	NETFLIX CASE STUDY (Q1)	AMAZON CHALLENGE (Q2)
Triggering Catalyst	2008 centralized database corruption causing a 3-day complete operational outage.	Early 2000s commercial growth gridlock, deployment trains, and frequent Obidos site crashes.
Architectural Shift	Monolithic application to cloud-native microservices on AWS with dedicated NoSQL stores.	Obidos monolith to decentralized Service-Oriented Architecture via the 2002 API Mandate.
DevOps Methodology	**Chaos Engineering:** Simian Army (Chaos Monkey, Gorilla, Kong) in production.	**Two-Pizza Teams:** *"You build it, you run it"* single-threaded ownership.
Resilience Mechanism	Circuit breakers (Hystrix), bulkheads, and graceful presentation degradation.	Cell-based architecture, automated canary analysis, and instant automated rollback (Apollo).
Key Outcome	**99.999% availability** across global regions supporting >260M subscribers.	Continuous deployment cadence averaging **one release every 11.7 seconds**.

5. ENGINEERING LESSONS & ACADEMIC CONCLUSION

The evolutionary trajectories of Netflix and Amazon demonstrate foundational principles of modern software engineering and DevOps:

- **Conway's Law is Inevitable:** High-performing microservice architectures cannot function within monolithic, siloed organizational structures. Amazon’s Two-Pizza teams and Netflix's culture of Freedom and Responsibility demonstrate that organizational restructuring (Reverse Conway Maneuver) is a mandatory prerequisite for architectural scalability.
- **Anti-Fragility over Fragile Perfection:** In planetary-scale cloud systems, failures are statistically certain. Netflix’s Chaos Engineering proves that intentionally injecting failure and designing for graceful degradation produces resilient, self-healing platforms.
- **Small Batch Size De-risks Delivery:** Releasing code in micro-batches (averaging every 11.7 seconds at Amazon) eradicates the catastrophic risk of "big-bang" releases, enabling continuous experimentation and sustainable competitive advantage.